ChatGPT

Autonomous ML Platform for Neurodegenerative Protein Aggregation Prediction

1. Multi-Disease, Multi-Dataset Platform Architecture

Design the data and experiment structure to avoid cross-contamination. **Namespacing:** Tag or partition experiments by disease/protein. For example, use separate MLflow *experiment* names or tags like `disease=alpha-synuclein` so results and models for Parkinson's do not mix with Alzheimer's runs. Store each disease's code/data in distinct modules or branches, or include a `--disease` config flag (e.g. via Hydra) to isolate flows.

Versioned Dataset Registry: Use data versioning tools (DVC, LakeFS, Git-LFS) to snapshot raw and processed datasets【70†L121-L129】【70†L150-L153】. Each dataset snapshot should have a unique ID (hash/digest) and be logged to an experiment registry (e.g. MLflow Data Tracking) along with metadata (source, schema)【90†L57-L64】. For example, MLflow's `log_input()` can record a dataset name and digest, ensuring **data lineage** and reproducibility【90†L57-L64】【70†L150-L153】. Always record *which version* of each dataset a run used, so that any result is tied to a specific data snapshot.

Unified Dataset Schema: Define a common format with core fields: e.g. `sequence` (amino-acid string), `protein_id`, `disease` (enum), `label` (aggregation class or score), plus experiment context (pH, temperature, assay type). Additional columns can capture predicted features (e.g. hydrophobicity, PTMs) or ontology terms. Disease-specific fields (e.g. Tau isoform, Synuclein mutation) can be optional columns or even separate tables. A useful guideline is the *Croissant* dataset standard, which advocates explicit structured metadata to make datasets reusable【72†L153-L156】. In practice, you might store all sequences in one table with `disease` as a key and use ontologies to harmonize labels (e.g. map different assays' scores to a common "aggregation severity" scale).

Transfer Learning: Leverage related datasets by pre-training and fine-tuning. In practice, pre-train a protein language model or neural network on one large dataset (e.g. Tau data) and then fine-tune on a smaller one (e.g. alpha-synuclein). Follow standard transfer-learning best practices: use a smaller learning rate on the target task, possibly freeze early layers or use adapters, and warm up training. The transfer-learning paradigm – "pre-train on one task, fine-tune on another" – is well established【74†L187-L195】. For example, one could fine-tune a model pre-trained on Alzheimer's tau-aggregation data to Parkinson's α-synuclein aggregation, assuming they share underlying features【74†L187-L195】. Always validate that performance improves over training from scratch on the target data.

2. Autonomous Experiment Execution Architecture

Implement an isolated, reproducible execution framework. **Self-contained scripts:** Each experiment proposal should generate a complete Python script or notebook (with dependencies declared) that can run independently. Use configuration frameworks (e.g. Hydra or argparse) so the script reads all

parameters from a config file or CLI and does not rely on interactive prompts. Avoid global state; instead, define a clear `main()` or class interface. For example:

```
class Experiment:
    def __init__(self, config): self.config = config
    def train(self):
        # build model, train on self.config data
    def evaluate(self):
        # compute metrics on validation data
    def run(self):
        self.train()
        metrics = self.evaluate()
        mlflow.log_metrics(metrics)  # record results
```

Every generated experiment script should implement a standard interface (e.g. `train()`, `evaluate()`, `save_results()`) so the orchestrator can uniformly invoke and log it.

Experiment Harness: Provide a runtime wrapper that calls each experiment's `run()` (or equivalent) method in a controlled way. Use an MLflow or similar experiment tracking API inside every script: call `mlflow.start_run()`, log all hyperparameters (`mlflow.log_params`) and metrics (`mlflow.log_metrics`), and save model artifacts (`mlflow.log_artifact` or `mlflow.pytorch.log_model`). Ensure the script logs a unique run ID and experiment name so outputs go to the persistent leaderboard. For example:

```
import mlflow
with mlflow.start_run(experiment_id="multi-disease"):
    mlflow.log_params(cfg.hyperparams)
    model = train_model(cfg)
    metrics = evaluate_model(model, cfg)
    mlflow.log_metrics(metrics)
    mlflow.pytorch.log_model(model, "model")
```

All relevant artifacts (trained model, plots, config files) should be saved to known locations so the orchestrator can ingest them.

Capturing Outputs: Structure outputs as JSON or Pandas DataFrames for metrics and logs, and upload them to the tracking system. For robust ingestion, write logs (including full training logs and exceptions) to files with unique names (`stdout.log`, `stderr.log`), and attach to the run record. Always record: dataset version IDs, random seed, all hyperparameters, environment (Python/library versions or container image ID), and git commit hashes. This metadata ensures full reproducibility of each run.

Failure Classification: Distinguish error types to handle them appropriately. Before execution, run a *static audit* (see next section) to catch syntax or import errors. During execution, wrap the training in try/except blocks. For example:

```
try:
    train()
```

```
except SyntaxError as e:
    mark_run("syntax_error", str(e))
except RuntimeError as e:
    if "out of memory" in str(e).lower():
        mark_run("oom_error", str(e))
    else:
        mark_run("runtime_error", str(e))
```

Label each failure (syntax, OOM, other runtime) and record it. If a run crashes without yielding metrics, mark it as failed and skip its results on the leaderboard. For "bad metric" (e.g. model returns NaNs or trivial accuracy), mark the run as unsuccessful and consider adjusting hyperparameters.

Retry and Skip Policies: Define a retry strategy for transient failures. For example, retry an OOM error with smaller batch size up to 2 times; retry network or disk errors a few times. After N retries (e.g. 3), abandon the experiment and log a failure. Always proceed to the next experiment in the queue to ensure continuity.

Experiment Deduplication: Before running a proposed experiment, check if an equivalent one has already been run. Compute a signature hash combining key settings (dataset version ID, model architecture name, and hyperparameter values). If this fingerprint matches a past run in the registry, skip execution. You can store a fingerprint map (e.g. MD5 of JSON config) for fast lookup to avoid redundant trials.

3. Leaderboard & Experiment Registry Design

Maintain a central results database (leaderboard) keyed by experiment. **Schema:** Each entry should include experiment ID, timestamp, disease/protein, dataset version IDs (for training/val/test splits), model/config identifiers, hyperparameters, random seed, and all evaluation metrics. Also record environment details (Docker image or conda env hash, library versions) and code commit hashes. For example, a JSON or database schema might have columns: `exp_id, disease, model_name, data_version, params_json, seed, metrics_json, timestamp, status`.

Reproducibility Metadata: Following ML reproducibility standards, log all details necessary to rerun an experiment: data version (hash/S3 URI), random seed(s), exact feature set, hyperparameters, model architecture/version, and software environment (python version, key library versions). The Croissant Task format emphasizes capturing compute environment and config as a blueprint for reproduction【72†L153-L156】. Use MLflow (or an equivalent) to attach this metadata to each run.

Comparing Across Versions: When datasets grow or change, maintain separate leaderboards per dataset version. To compare results across versions, you can use relative metrics: e.g. compare improvement over a baseline model on the *same* version, rather than raw accuracy. Optionally re-evaluate top models on the new data to benchmark consistency. Normalization strategies include reporting performance deltas or using percentage of best-known performance. Ensure the leaderboard records the dataset version to interpret scores correctly.

Statistical Significance: To judge if one experiment truly outperforms another, perform significance tests on the metric distributions. For example, run each model configuration several times and use paired statistical tests (paired t-test or Wilcoxon signed-rank) on the key metric (e.g. macro-F1) to check if differences are beyond chance. For classification, a McNemar's test or bootstrap confidence interval

on accuracy/F1 can be used. Only declare a new result superior if the confidence intervals do not overlap or a p-value is below a threshold (e.g. 0.05), accounting for multiple comparisons if needed.

4. Continual / Incremental Learning for Growing Datasets

Plan for evolving data. **Retraining vs Incremental:** When new labeled data arrives, you have options: (a) fully retrain the model on all data (most robust but costly); (b) fine-tune the existing model on only the new data (faster but risks forgetting old patterns); (c) use *replay buffers* where a subset of old examples is mixed with new data during fine-tuning【41†L85-L92】. In practice, a hybrid approach often works: periodically retrain from scratch (e.g. monthly) and in between perform incremental fine-tuning with replay.

Catastrophic Forgetting: Deep networks often forget previous knowledge when fine-tuned on new tasks. Use known continual learning techniques: (i) *replay methods* (keep exemplars of old data), (ii) *parameter regularization* (e.g. Elastic Weight Consolidation to constrain important weights), or (iii) *knowledge distillation* (fitting the new model to mimic the old one's outputs on old data). Some architectures are naturally more robust (e.g. smaller shallow nets or ensemble methods can be less prone to forgetting than very deep nets). Practically, continual learning research offers six families of methods (replay, regularization, functional regularization, optimization tricks, context-specific, template-based)【41†L85-L92】. Tools like PyTorch-EWC or distillation fine-tuning can help implement these.

Shift Detection: Automatically check if new data has shifted distribution from training data. Monitor changes in class proportions or feature statistics (mean, variance). For example, track the distribution of aggregation labels or key sequence features (lengths, amino-acid frequencies) in incoming batches. Use drift-detection libraries: *Frouros* provides many concept- and data-drift detectors out of the box【47†L314-L322】. Also consider *Evidently AI* or other monitors: you could, for example, compute a two-sample test (KS-test) on feature distributions, or monitor model prediction distributions. If metrics (like validation accuracy) degrade on new data, that also signals shift【45†L68-L77】.

Retraining Triggers: Decide whether to retrain on schedule or event-driven. A hybrid approach works well: (a) **Event-driven:** if drift is detected above a threshold or a drop in validation score occurs, trigger a retraining pipeline. (b) **Scheduled:** e.g. monthly retrains even if drift is small, to ensure up-to-date knowledge. Automate this with a workflow engine (e.g. Airflow/Kubeflow) that either polls for new data or is triggered by data arrivals.

Libraries: Use specialized continual-learning and online-learning libraries. *Avalanche* (PyTorch) is an end-to-end continual learning library for prototyping and evaluation【43†L84-L93】. It provides benchmarks, training loops, and many CL algorithms. For streaming data scenarios, *River* (formerly Creme) offers online learning algorithms. And as noted, *Frouros* helps detect drift automatically【47†L314-L322】. Combining these with your training code can accelerate incremental learning research.

5. Variable-Length Sequence Modeling

Handle sequences of arbitrary length elegantly. **Model Architectures:** Use sequence models that natively accept variable lengths. Common choices are RNNs (LSTM/GRU), 1D CNNs with global pooling, or Transformer-based protein language models. Attention-based models (BERT-like or ProtBERT/ESM) can process any length (within memory limits) by using masks. For example, *ProteinBERT* introduced local/global heads that allow end-to-end processing of long protein sequences by combining per-

residue and whole-sequence embeddings【55†L147-L156】. In general, Transformers (ESM-1b/ESM-2, ProtT5, ProtAlbert) and LSTMs can handle variable lengths without needing fixed windows.

Fixed-Size Representation: After encoding, aggregate to a fixed-size feature vector for classification/regression. Typical strategies: take the [CLS] token embedding (if using BERT-like model), or apply mean/max pooling over token embeddings. For LSTMs, use the final hidden state or attention pooling. Explicit pooling layers ensure the model output does not depend on sequence length. You can also use learned attention-pooling (self-attention over the sequence) to compute a weighted average of token embeddings.

Protein Language Models: Leverage pre-trained protein LMs as feature extractors. Good choices include Facebook's ESM-2 models (various sizes), ProtT5, ProtBERT, or ProtXLNet. For short-to-medium peptides, even smaller models (ESM2 8M or PepBERT) can be effective. For aggregation specifically, note that the *PALM* model uses ESM2-8M embeddings with a custom light-attention head to predict aggregation propensity【52†L269-L277】, demonstrating that fine-tuned LMs excel at these tasks. Evaluate several (e.g. `facebook/esm2_t6_8M_UR50D`, `Rostlab/prot_bert`) and consider ensembling if data permits.

Padding vs Masking: Pad sequences to the same length within a batch, but always use mask tokens so padding does not contribute to attention or pooling. In Transformer models, supply an attention mask array indicating real tokens. Avoid naive zero-padding without masking, as it can bias pooled representations. Alternatively, pack sequences (for RNNs) and remember their original lengths. In any fixed-dimension encoder, ensure padded positions are ignored in pooling (e.g. by summing only over `length` positions).

Sliding-Window Approach: If sequences exceed model input limits, use overlapping sliding windows. Split each long peptide into chunks (e.g. length 512 with stride 256), run each chunk through the model, and then aggregate the predictions (e.g. take the maximum aggregation score across windows). However, beware leakage: do not let windows from the same peptide cross train/test splits (see Section 12). Sliding windows are mainly useful when patterns are highly local; if the model can handle whole sequences (with efficient attention or sparse attention), that is preferable.

6. Multi-Agent Orchestration for Long-Running Research Platforms

【67†embed_image】
Implement a multi-agent loop where different "agents" handle planning, coding, auditing, execution, and evaluation. A high-level *Planner* (LeadResearcher) agent formulates hypotheses or next experiments, then delegates to sub-agents. For example, the LeadResearcher might save its reasoning into a persistent memory and spawn a *LiteratureAgent* to fetch relevant papers【18†L129-L137】. It reviews those findings, then possibly spawns an *ExecutionAgent* to run experiments. This loop continues until the Planner decides enough information is gathered【18†L129-L137】. Frameworks like Microsoft's AutoGen or LangChain can orchestrate such multi-agent systems【78†L245-L254】. By distributing tasks (Planner proposes tasks, Coder writes code, Auditor checks it, Executor runs it, Evaluator reviews metrics), the system mimics a research team. In practice, you would implement clear interfaces between roles (e.g. Planner outputs a specification that Coder must implement) and use a shared *memory* (database) for agents to read/write intermediate results.

【19†embed_image】
Agents share state via a persistent memory and tool interfaces. For instance, all experiment proposals and results go into a database that the Planner can query to avoid repeating old ideas. Sub-agents access external tools: literature search (PubMed API), code execution (sandboxed Python), static analysis (AST linter), and model training environments. Multi-agent research can reduce errors and hallucinations, since agents check each other's work【78†L202-L210】. Because this platform runs over months, implement checkpointing: periodically serialize the agent queue, memory, and experiment state so the system can resume after crashes. Also use asynchronous execution to parallelize: while one agent is running an experiment, the Planner can already draft the next hypothesis.

7. Typed Tool Registry Design

Define a strict API for every tool the agents can call. For example:

- `propose_experiment(disease: str) -> ExperimentSpec`
- `generate_code(spec: ExperimentSpec) -> CodeFile`
- `audit_code(code: CodeFile) -> AuditReport`
- `run_experiment(code: CodeFile) -> ExperimentResult`
- `log_to_leaderboard(result: ExperimentResult) -> None`
- `fetch_literature(query: str) -> List[Document]`
- `register_dataset(meta: DatasetMeta) -> DatasetID`
- `detect_drift(data: Dataset) -> DriftReport`

Use Python dataclasses or Pydantic models to enforce input/output types and validate schema. Version each tool (e.g. v1.0.0) so agents refer to immutable interfaces and can fall back if implementations evolve. For backward compatibility, allow calling older tool versions.

Static Code Auditing: Before running agent-generated code, perform AST-based checks. For example, parse the code with `ast.parse` and scan for forbidden patterns (like `os.system` or file I/O outside allowed paths). Tools like `pylint`, `flake8`, or security scanners (Bandit) can flag syntax errors or bad practices. This audit step should block execution of any code that fails basic correctness or safety rules.

Sandboxing Generated Code: Execute all code in isolated containers or processes. As recommended for AI-generated code, use an *ephemeral coding sandbox*: a fresh container or VM with no network access (deny egress), a limited filesystem, and strict CPU/memory quotas【39†L105-L113】. For instance, spawn a Docker container per experiment with the experiment script and data volume mounted read-only. Enforce a timeout and kill the container after completion. This protects the shared cluster from rogue or infinite-loop code, as the sandbox auto-destroys【39†L105-L113】.

8. Protein Embedding & Feature Toolchain

Construct rich input features from sequences and known predictors. **Protein Language Models:** Use state-of-the-art PLMs to embed sequences. Suitable checkpoints include Facebook's ESM2 series, ProtBERT/T5, or smaller models like ESM2-8M for efficiency. For short peptides, consider models fine-tuned on peptides (e.g. PepBERT). These models output contextualized embeddings (per residue and/or pooled). For instance, the PALM project combined ESM2 (8M) embeddings with a light-attention head to predict amyloidogenic regions【52†L269-L277】, showing the effectiveness of ESM for aggregation tasks.

Aggregation Predictors: Compute established aggregation propensity scores as additional features. Tools such as TANGO, WALTZ, CamSol, Zyggregator, AGGRESCAN, or PASTA produce scalar scores or per-residue profiles. For example, TANGO predicts beta-sheet-mediated aggregation tendencies, and Zyggregator uses amino-acid propensities – these are "phenomenological" algorithms in the literature【84†L452-L460】. Another class of methods (including TANGO and Waltz) estimates peptide-specific aggregation propensity patterns【84†L459-L464】. Many predictors have published software or web APIs; integrate them as featurizers (e.g. call a Python port of CamSol, or lookup WALTZ scores). Include their outputs as numeric features or one-hot exceedence flags.

PTM Encoding: Represent post-translational modifications (acetylation, phosphorylation, ubiquitination, etc.) explicitly. Options: extend the amino-acid alphabet (e.g. treat phospho-serine as a special token), or add parallel indicator features (e.g. a boolean vector aligned to the sequence marking PTM sites). Some studies embed PTMs by adding extra bit vectors to each residue's input embedding. Ensure the model can distinguish modified vs. unmodified residues.

Unified Feature Vector: Concatenate sequence-derived features into one input tensor. For example, take the PLM's [CLS] or pooled sequence embedding, then append additional features: the sequence length, computed aggregation scores (TANGO, etc.), hydrophobicity indices, and any PTM indicators. This composite vector then goes into the prediction head. Experiment with joint-training the model on these features or feeding them through a small MLP concatenated with the PLM output.

9. Imbalanced Multi-Class Classification Across Varying Dataset Sizes

Adapt to class imbalance and changing data sizes. **Re-sampling vs. Re-weighting:** For severe imbalance, you can oversample minority classes (e.g. duplicate or augment rare peptides) or undersample the majority. More robust is to use *class-weighted loss functions*: in PyTorch/CrossEntropy, supply `class_weight = 1/(class_freq)` so rare classes incur higher loss. Alternatively, use *focal loss* (a variant of cross-entropy that down-weights easy examples) to focus training on the minority classes. Compare these experimentally. As data grows, re-weighting usually scales automatically, whereas oversampling may become inefficient.

Ordinal Loss: If aggregation severity is ordinal (e.g. "none < mild < high"), consider ordinal regression losses. For instance, discretize severity and use a weighted penalty that penalizes larger class-difference more. The "ordinal logistic loss" or treating labels as a rank order can capture this structure. If labels are continuous (aggregation amount), treat it as regression or order the classes.

Metrics: Use evaluation metrics robust to imbalance. Macro-averaged metrics (macro-precision/recall/F1) treat each class equally, avoiding domination by the majority. Balanced accuracy (mean recall) and Cohen's Kappa (chance-adjusted agreement) are also good. Always report multiple metrics (e.g. macro-F1 and per-class recall) so the leaderboard remains meaningful at small N and scales as data grows.

10. Literature Integration for Autonomous Research

Automate fetching and using relevant papers. **API Integration:** Use the NCBI Entrez/PubMed API (e.g. via Biopython or `pymed`) to query new literature by protein name and keywords【88†L89-L95】. For example, use `pymed.PubMed().query("tau aggregation", max_results=100)` to retrieve recent abstracts【88†L89-L95】. Similarly, bioRxiv offers a REST API (`api.biorxiv.org`) to query preprints by date or category【58†L16-L25】【58†L58-L66】. For instance, pull all new "bioRxiv"

preprints in the “neuroscience” category or matching “alpha-synuclein”. Collect titles and abstracts automatically on a schedule.

Knowledge Extraction: Summarize or extract key points from long abstracts without overflowing context. A good approach is to chunk each abstract into paragraphs or sentences (≤200 tokens) and embed them using a language model (e.g. SentenceTransformer). Use retrieval (RAG) to find the most relevant chunks for a given hypothesis. For example, store all abstract chunks in a vector database (FAISS/Chroma) with embeddings, then at hypothesis time retrieve the top-K matching segments. You can also use an LLM to summarize abstracts into bullet points (few-shot prompting) so that the agent’s memory retains concise knowledge.

RAG and Chunking: When building a RAG pipeline, chunk at natural sentence or paragraph boundaries to preserve meaning. A common practice is 256–512 token chunks with some overlap. Index them with a transformer embedding model (e.g. `all-MiniLM-L6-v2`). During planning, the Planner agent queries this knowledge base with keywords or draft queries. The retrieved context is then fed into the LLM either by concatenating with the prompt or via retrieval calls. This ensures that literature context informs hypothesis generation without exceeding LLM input limits.

Feeding Context: Pass literature findings to the Planner by including relevant abstracts or summaries in the prompt. For instance, “Based on recent findings: [abstract excerpt], [abstract excerpt]..., suggest a new experiment.” This grounding helps the Planner propose experiments aligned with the latest research. Maintain a cache of key sentences to avoid repeated API calls, and update the memory when new papers arrive.

11. Dataset Quality & Label Consistency Across Disease Expansion

Ensure data integrity as you add new diseases. **Unified Label Ontology:** Define a common scale for “aggregation severity” across proteins. For example, classify aggregation propensity as {0: None, 1: Mild, 2: Moderate, 3: Severe} or use normalized continuous scores. Map diverse assay readouts (ThT fluorescence, turbidity) onto this ontology via thresholds or rank-percentiles. Document any disease-specific mapping rules. This allows cross-disease comparisons and multi-task learning on a unified label space.

Data Validation: Rigorously check new data before ingestion. Use libraries like *Great Expectations* to define assertions on schema and values【63†L80-L84】. For example, validate that every record has a valid sequence (only A-Z letters), a disease code from a controlled vocabulary, label within expected range, and no missing critical fields【63†L80-L84】. Automate these checks as part of data ingestion: any anomalous entries (wrong format, out-of-range labels) should be flagged or rejected.

Handling Conflicting Labels: It may happen that the same peptide sequence is reported with different labels under different conditions. In such cases, keep entries separate but record the experimental context (temperature, pH, concentration) as part of the metadata. You can treat each record as a unique (sequence, context) instance. If true conflicts (same context but different labels), mark them and possibly exclude from training. Track agreement between duplicate measurements: if replicates differ, you might take the average or use probabilistic labels.

Inter-Annotator Agreement: As new experimental labels come in from wet-lab sources, track consistency. If multiple lab results are available for the same sequence, compute inter-annotator

metrics like Cohen's kappa or Fleiss's kappa on class labels to quantify agreement. A high level of disagreement may indicate noisy data. Log these statistics periodically to monitor data quality trends.

12. Evaluation Protocol Design

Prevent leakage and ensure fair comparison. **Correct Metrics:** For imbalanced multi-class problems, use metrics insensitive to class frequency: macro-F1 or macro-averaged recall, balanced accuracy, or Cohen's Kappa (especially for ordinal labels). Include both per-class and average scores on the leaderboard. For regression tasks, report metrics like mean absolute error (MAE) and R^2, but also consider median error if outliers exist.

Locked Evaluation Oracle: Use a hidden test set that the agent cannot query. For example, hold out a final test subset (possibly from the latest data) and only evaluate against it in closed loop, similar to a Kaggle or MLPerf private leaderboards. The agent should log predictions on this set, and a separate evaluation service computes scores. This prevents the agent from overfitting to the test labels. The evaluation oracle code (which calculates final metrics) should be static and not accessible to the agent.

Preventing Data Leakage: Split data at the *peptide* (or protein) level, not at window level. In sliding-window scenarios, first split sequences into train/val/test groups, then generate windows. This ensures that no overlapping sequence segments appear in both train and test. Leakage can severely inflate performance; as one study notes, creating windows before splitting lets future information leak into training【66†L293-L300】. In our pipeline, ensure that sequence windows are constructed only after splits are done, maintaining chronological or group integrity【66†L293-L300】.

Peptide- vs Window-Level Splitting: Be explicit about split granularity. If a protein has multiple overlapping fragments (windows), all fragments from one parent peptide should stay in the same partition. For example, use a grouped K-fold by peptide ID. Following 【66†L293-L300】, generating sliding windows *after* data partitioning is essential: only then can we guarantee no leakage. Document this rule in the evaluation code. This way, test metrics truly reflect generalization to new peptides, not memorization of overlapping regions.

Secure Leaderboard: Finally, to prevent the agent from gaming metrics (e.g. by overfitting to known validation performance), consider keeping the true test labels offline. Have the agent submit predictions and only return scores. Rotate test sets periodically to ensure robustness. This locked evaluation protocol ensures a fair and meaningful leaderboard that can stand the test of time.

References: We rely on modern MLOps practices and current research. For example, data versioning tools and dataset tracking are critical for reproducibility【90†L57-L64】【70†L150-L153】. Continual learning literature provides strategies to avoid forgetting【41†L85-L92】. Safe sandboxing principles are informed by best practices in agent-generated code【39†L105-L113】. Key protein modeling resources (e.g. ESM, ProtBERT) and aggregation predictors (TANGO, WALTZ, Zyggregator) inform our feature design【52†L269-L277】【84†L452-L460】. Multi-agent research frameworks (Anthropic, AutoGen, LangChain) guide the orchestration architecture【18†L129-L137】【78†L245-L254】. All tools and methods mentioned above have open-source implementations or documentation, ensuring practical feasibility.
